WandB Tracking Reference

GRPO Fine-tuning for Code Generation · Qwen2.5-Coder-7B

logging_steps=1 | eval every 200–300 steps | G=4 group size

Legend:

Green = TRL auto-logged	Yellow = Key watch metric	Red = Alert condition	White = Custom logged in reward_fn

1. Training Dynamics

Automatically logged by TRL. No custom code needed. Covers core optimization health.

Metric Key	Description	Frequency	Source
train/loss	Primary training loss	Every step	TRL auto
train/learning_rate	LR scheduler value	Every step	TRL auto
train/grad_norm	Gradient norm — spikes indicate instability	Every step	TRL auto
train/global_step	Current training step	Every step	TRL auto
train/epoch	Current epoch	Every step	TRL auto

2. Reward Signals

Logged inside reward_fn. Requires difficulty metadata flowing through from dataset via kwargs. Batch means logged per step.

Metric Key	Description	Frequency	Notes
Overall			
reward/mean	Mean reward across batch	Every step	Primary training signal
reward/std	Reward std across batch	Every step	Should decrease over time as model stabilizes
reward/non_zero_fraction	Fraction of batch with reward > 0	Every step	Key sparsity indicator — watch early training
By Component			
reward/execution_mean	Mean sandbox execution score	Every step	Backbone signal (0.6–0.75 weight)
reward/reasoning_mean	Mean reasoning quality score	Every step	Gemini judge + presence (0.25–0.40 weight)
By Difficulty			
reward/execution_easy	Execution score, easy problems	Every step	Should plateau early — model masters easy
reward/execution_medium	Execution score, medium problems	Every step	Should climb through phase 2

reward/execution_hard	Execution score, hard problems	Every step	Should move only in phase 3+
reward/reasoning_easy	Reasoning score, easy problems	Every step	Presence heuristic dominates
reward/reasoning_medium	Reasoning score, medium problems	Every step	70% Gemini — most reliable signal
reward/reasoning_hard	Reasoning score, hard problems	Every step	30% Gemini — dampened, execution dominates

3. GRPO Diagnostics

Catches the three core GRPO failure modes: sparse signal (all_zero), saturated signal (all_correct), and degenerate advantages (reward_std). Standard metrics auto-logged by TRL; degenerate group metrics require custom computation in reward_fn.

Metric Key	Description	Frequency	Alert Condition
Standard GRPO (TRL auto)			
grpo/kl_divergence	KL from reference policy	Every step	🚨 Spike = model collapsing, stop training
grpo/clip_fraction	How often PPO clip activates	Every step	High fraction = too large policy updates
grpo/advantage_mean	Mean advantage across groups	Every step	Should be near 0 (normalized)
grpo/advantage_std	Std of advantages across groups	Every step	Near 0 = degenerate groups
grpo/entropy	Policy entropy	Every step	Drop to 0 = model stopped exploring
Degenerate Group Detection (custom)			
grpo/all_zero_fraction	Groups where all G=4 rewards = 0	Every step	High = problems too hard, sparse signal
grpo/all_correct_fraction	Groups where all G=4 rewards = 1	Every step	High = problems too easy, saturated signal
grpo/degenerate_fraction	all_zero + all_correct combined	Every step	🚨 > 0.8 = learning stalled
grpo/reward_std_mean	Mean std within each G=4 group	Every step	🚨 < 0.05 = advantages near zero everywhere

```
Degenerate group computation (inside reward_fn, after scores computed):
groups = [rewards[i:i+G] for i in range(0, len(rewards), G)]
wandb.log({'grpo/all_zero_fraction': mean([all(r==0 for r in g) for g in groups]), ...
```

4. Generation Quality

Tracks output format health. Parsed from raw completions in reward_fn before sandbox execution. Split by difficulty using the same mask pattern as reward signals.

Metric Key	Description	Frequency	What to Watch
gen/response_length_mean	Mean token length of completions	Every step	Sudden drop = model truncating early
gen/steps_count_mean	Mean number of [STEP N] blocks	Every step	Floor of 3 enforced by presence reward
gen/steps_count_easy	Mean steps, easy problems	Every step	Should naturally be lower than hard
gen/steps_count_medium	Mean steps, medium problems	Every step	Should increase as model learns structure
gen/steps_count_hard	Mean steps, hard problems	Every step	Should be highest tier
gen/valid_code_fraction	Fraction of outputs with parseable ```python block	Every step	🔒 Drop = model forgetting code format
gen/valid_code_easy	Valid code fraction, easy	Every step	Should be high throughout
gen/valid_code_medium	Valid code fraction, medium	Every step	Watch for drops on harder problems
gen/valid_code_hard	Valid code fraction, hard	Every step	Lowest tier — acceptable if rising

5. Curriculum Tracking

Confirms phase transitions are firing at correct steps and actual sampling fractions match config. Phase is derived from global_step — no stateful tracking needed.

Metric Key	Description	Frequency	Expected Values
curriculum/current_phase	Active phase (0–3)	Every step	0→1 at step 300, 1→2 at 800, 2→3 at 1500
curriculum/easy_fraction	Actual easy sampling fraction	Every step	1.0 → 0.5 → 0.3 → 0.2 across phases
curriculum/medium_fraction	Actual medium sampling fraction	Every step	0.0 → 0.5 → 0.5 → 0.4 across phases
curriculum/hard_fraction	Actual hard sampling fraction	Every step	0.0 → 0.0 → 0.2 → 0.4 across phases

6. Evaluation (Pass@1)

Expensive to compute — run full LCB eval (713 problems) every 200–300 steps only. This is the primary external validity metric and the number that goes on your resume.

Metric Key	Description	Frequency	Notes

eval/pass@1_lcb	Pass@1 on full LiveCodeBench (713 problems)	Every 200–300 steps	Primary resume metric — delta vs baseline
eval/pass@1_easy	Pass@1, easy LCB subset	Every 200–300 steps	Should be highest and plateau early
eval/pass@1_medium	Pass@1, medium LCB subset	Every 200–300 steps	Key improvement signal
eval/pass@1_hard	Pass@1, hard LCB subset	Every 200–300 steps	Lower absolute value, watch the trend

7. Healthy Training Reference

Expected metric ranges at each training phase. Use as a quick sanity check during runs.

Metric	Early Training	Mid Training	Late Training
reward/non_zero_fraction	~0.2–0.3	~0.4–0.6	>0.6
grpo/reward_std_mean	~0.1–0.2	~0.2–0.3	~0.2–0.3
grpo/degenerate_fraction	~0.5–0.6 (ok)	~0.3–0.4	<0.3
grpo/kl_divergence	Low, stable	Low, stable	⚠ Watch for spikes
gen/valid_code_fraction	>0.7	>0.8	>0.85
reward/execution_medium	~0.1–0.2	~0.3–0.5 (climbing)	>0.5

8. Alert Conditions

Stop or investigate immediately if any of these are observed.

Condition	Action
grpo/kl_divergence spikes sharply	Stop training immediately. Reduce learning rate or increase KL penalty coefficient.
gen/valid_code_fraction < 0.5	Model is forgetting code format. Check prompt template, consider reducing LR.
grpo/degenerate_fraction > 0.8 sustained	Learning stalled. Adjust curriculum phase thresholds or check sandbox.
grpo/reward_std_mean < 0.05 sustained	Advantages near zero. Check if all problems in a batch are same difficulty.
reward/non_zero_fraction < 0.1 after step 200	Reward too sparse. Check sandbox execution, verify test case format.
train/grad_norm explodes	Gradient clipping not working. Check max_grad_norm config.

All per-step metrics use logging_steps=1 in GRPOConfig. Eval metrics logged separately via custom eval loop called every eval_steps.